

Gra karciana sterowana gestami

Sprzętowa implementacja Blackjacka na FPGA
z rozpoznawaniem gestów przez czujnik IMU LSM6DSO

Bartłomiej Raczyński (BR)

Eryk Rutka (ER)

15 czerwca 2026

SystemVerilog | Basys3 FPGA | VGA | SPI

Spis treści

1	Repozytorium git	2
2	Wstęp	2
3	Specyfikacja	2
3.1	Opis ogólny algorytmu	2
3.2	Tabela zdarzeń	4
4	Architektura	6
4.1	Moduł: top	6
4.1.1	Schemat blokowy	6
4.1.2	Porty	7
4.1.3	Interfejsy	8
4.2	Rozprowadzenie sygnału zegara	9
5	Implementacja	10
5.1	Lista zignorowanych ostrzeżeń Vivado	10
5.2	Wykorzystanie zasobów	11
5.3	Marginesy czasowe	12
6	Konfiguracja sprzętu	12
6.1	Schemat połączenia płytek Basys3 w trybie multiplayer	12
6.2	Podłączenie urządzeń peryferyjnych	13
6.3	Konfiguracja przełączników	13
7	Film	13

1 Repozytorium git

Adres repozytorium:

https://github.com/branchynski/Bjack_with_gesture_control

2 Wstęp

Projekt polega na stworzeniu sprzętowej implementacji gry w Blackjacka uruchamianej na układzie FPGA. Innowacyjnym elementem projektu jest zastąpienie klasycznego sterowania przełącznikami przez system rozpoznawania gestów.

Czujnik IMU:	LSM6DSO (akcelerometr + żyroskop)
Interfejs VGA:	Kontroler VGA (1024 x 768)
Język HDL:	SystemVerilog + (Verilog z VITIS)
Platforma:	Basys3 (Xilinx Artix-7)

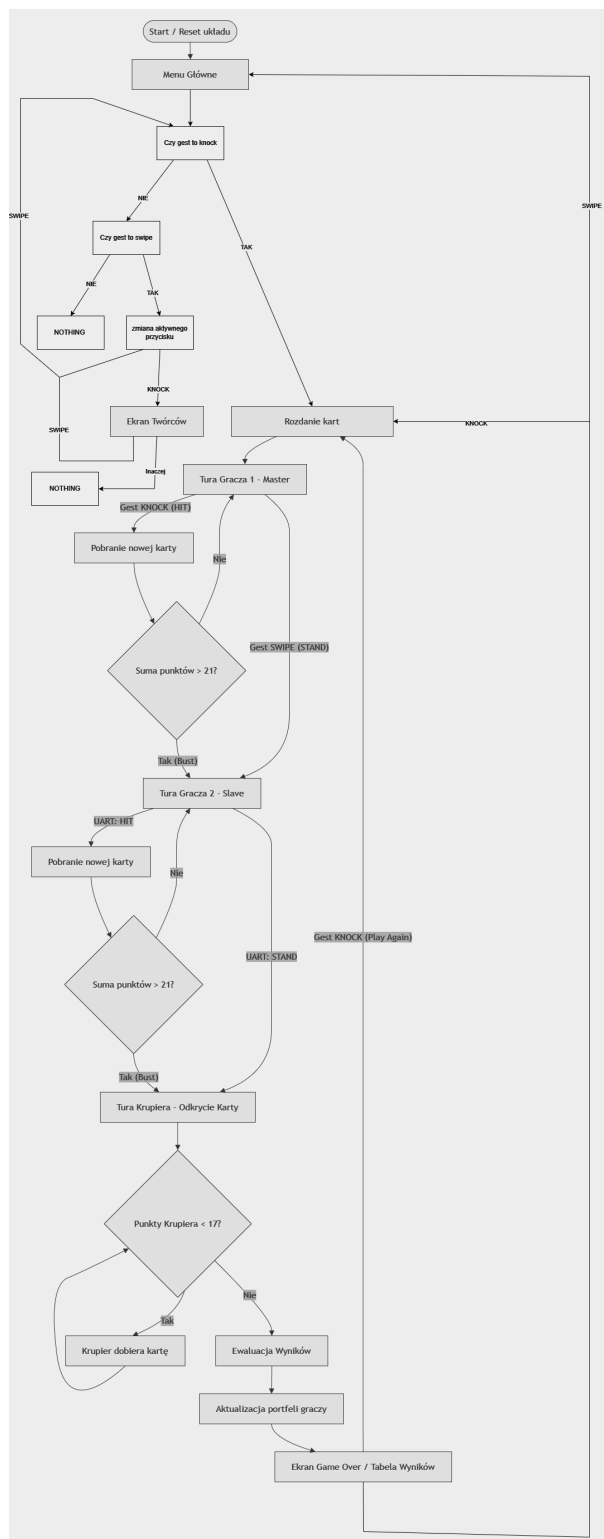
Dane z czujnika IMU są na bieżąco analizowane i mapowane na decyzje w grze. Interfejs użytkownika (stół, karty, wyniki) jest sprzętowo generowany i wyświetlany na monitorze przez kontroler VGA.

3 Specyfikacja

3.1 Opis ogólny algorytmu

Po uruchomieniu układu inicjalizowany jest generator sygnałów zegarowych, a na ekranie podłączonym przez port VGA pojawia się ekran startowy.

Projekt składa się z dwóch głównych części: gry Blackjack i kontrolera gestów. Poniżej przedstawiono algorytm gry.



Algorytm detekcji gestów

Algorytm realizuje potokowe przetwarzanie danych z czujnika inercyjnego (IMU) w celu detekcji i klasyfikacji gestów (przesunięcie dłonią – *swipe*, oraz pukanie – *knock*) przy użyciu sprzętowego akceleratora na układzie FPGA. Jego działanie dzieli się na cztery główne etapy:

1. Akwizycja i buforowanie danych z przesuwным oknem (*Sliding Window*)

Sześcioosiowe dane (akcelerometr i żyroskop) pobierane są z czujnika poprzez interfejs SPI i na bieżąco wpisywane do bufora o stałej wielkości okna $W = 208$ próbek. Mechanizm przesuwania okna aktywuje się co określoną liczbę nowych pomiarów (krok $S = 20$). W momencie wyzwolenia, wskaźnik odczytu zostaje zamrożony (`wr_ptr_snap`), a moduł rozpoczyna bezkolizyjną transmisję 208-elementowej ramki po magistrali AXI-Stream do akceleratora. Dzięki temu bufor może w tle, bez przerywania pracy, wciąż przyjmować i zapisywać napływające, nowe próbki z IMU.

2. Skalowanie sprzętowe

Aby dopasować zakres danych wejściowych do sprzętowej arytmetyki stałoprzecinkowej `ap_fixed<16,6>` (obsługującej zakres od -32.0 do 31.999), odczyty surowe poddawane są dzieleniu przez współczynnik 1024. W strukturze sprzętowej operacja ta nie wymaga użycia kosztownych dzielników sprzętowych – realizowana jest w sposób wysoce zoptymalizowany, logicznie odpowiadając bitowemu przesunięciu w prawo ($\gg 10$).

3. Inferencja za pomocą sieci 1D-CNN

Ramka danych trafia do zaimplementowanego w języku HLS akceleratora głębokiej sieci neuronowej. Wejściowa macierz o wymiarach 208×6 podlega operacjom splotu jednowymiarowego (Conv1D), normalizacji (*Batch Normalization*), nieliniowej aktywacji (ReLU) oraz redukcji wymiarów (*MaxPooling1D*). Wyjściem warstw gęstych (*Dense*) są trzy surowe wagi (logity) spakowane w pojedyncze 48-bitowe słowo wyjściowe, odpowiadające prawdopodobieństwom trzech klas: tła (brak gestu), *swipe* oraz *knock*.

4. Maszyna stanów (FSM) i kontrola wyników

Za filtrację wyników i niwelowanie szumu decyzyjnego odpowiada moduł `model_controller_fsm`, który implementuje następujące reguły:

- **Margines pewności:** Logit wskazujący na gest musi bezwzględnie przewyższać wartość wyliczoną dla tła o stały współczynnik (`CONFIDENCE_MARGIN = 300`) oraz być większy od wyniku drugiego z gestów.
- **System głosowania:** Weryfikacja opiera się na integracji wyników w czasie. System zlicza przewagi w kolejnych cyklach inferencji. Finalna decyzja o wystąpieniu gestu zapada dopiero, gdy określona akcja zbierze 5 głosów (`VOTE_THRESHOLD`).
- **Blokada czasowa (*Flush window*):** Zgłoszenie udanej detekcji na wyjściu układu inicjuje odliczanie. System celowo ignoruje wyjścia z sieci przez kolejnych 20 inferencji (`FLUSH_INFERENCES`), zapobiegając wielokrotnemu zarejestrowaniu długotrwałego ruchu jako oddzielnych, niezależnych gestów.

3.2 Tabela zdarzeń

Opis zdarzeń występujących podczas działania urządzenia – zarówno zewnętrznych (interakcje z użytkownikiem), jak i wewnętrznych (specyficzne stany algorytmu).

Zdarzenie	Kategoria	Reakcja systemu
Podwójne puknięcie	Menu główne	Wciśnięcie przycisku.
Przesunięcie w prawo	Menu główne	Zmiana aktywnego przycisku.
Wygenerowanie widoku z autorami	Ekran twórców	Wyświetlenie okna informacyjnego.
Przesunięcie w prawo	Ekran twórców	Przejsie do menu głównego.
Wylosowanie kart dla krupiera i graczy	Rozdanie kart	Wyświetlenie kart na ekranie.
Podwójne puknięcie	Tura gracza 1	Dobranie karty.
Suma punktów przekracza 21	Tura gracza 1	Przejsie do tury gracza drugiego (BUST).
Przesunięcie w prawo	Tura gracza 1	Przejsie do tury gracza drugiego.
Podwójne puknięcie	Tura gracza 2	Dobranie karty.
Suma punktów przekracza 21	Tura gracza 2	Przejsie do tury krupiera (BUST).
Przesunięcie w prawo	Tura gracza 2	Przejsie do tury krupiera.
Liczba punktów mniejsza niż 17	Tura krupiera	Dobranie karty.
Liczba punktów większa lub równa 17	Tura krupiera	Ewaluacja wyników.
Podwójne puknięcie	Menu końcowe	Ponowne zagranie.
Przesunięcie w prawo	Menu końcowe	Powrót do menu głównego.
Odczyt i zapis nowej próbki danych z IMU	Akwizycja i pamięć	Inkrementacja wskaźnika zapisu w pamięci BRAM oraz podbicie licznika kroków przesunięcia (<code>step_cnt</code>).
Osiągnięcie liczby próbek równej krokowi ($S = 20$)	Sterowanie przepływem	Skopiowanie aktualnego wskaźnika jako bezpiecznej bazy (<code>wr_ptr_snap</code>) i inicjalizacja rzutu zawartości całego okna w postaci ramki AXI-Stream.
Odebranie słowa wyjściowego z układu HLS	Walidacja logitów	Przesłanie zarejestrowanych wyników przez komparatory z zachowaniem marginesu pewności i przyznanie 1 głosu na dominującą klasę decyzyjną.
Skumulowanie 5 głosów (<code>VOTE_THRESHOLD</code>) dla danej klasy	Zdarzenie decyzyjne	Zmiana sygnału wyjściowego na zidentyfikowany gest (SWIPE lub KNOCK), wyzerowanie systemu głosów i rozpoczęcie odliczania czasu martwego.
Otrzymanie wyniku przy aktywnym liczniku <code>flush_cnt</code>	Filtracja (czas martwy)	Wyniki z akceleratora zostają całkowicie zignorowane, proces głosowania zawieszony, następuje wyłącznie dekrementacja licznika <code>flush_cnt</code> .
Odliczanie <code>flush_cnt</code> osiąga wartość zerową	Powrót do gotowości	Zresetowanie wyjścia systemu do stanu bazowego (NOTHING) i wznowienie nasłuchu ze wznowianiem przyjmowania głosów od akceleratora.

4 Architektura

⚠ Uwaga

Dobrze zrobiony projekt zawiera tylko moduły **strukturalne** (zbudowane z innych modułów) i **funkcjonalne** (zawierające bloki proceduralne **always @**). Staramy się nie generować bloków mieszających te dwa typy – głównym kryterium powinna być czytelność kodu.

Opisujemy architekturę tylko **głównego modułu** oraz **rozprowadzenie sygnału zegara**.

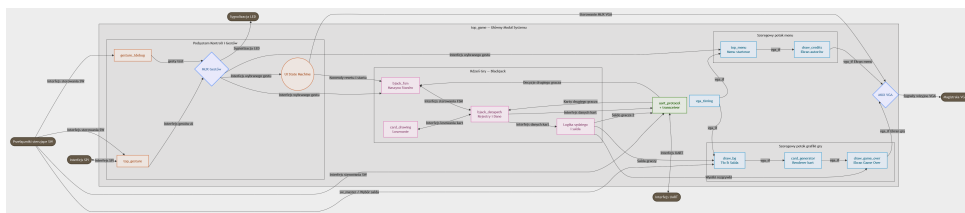
4.1 Moduł: top

Osoba odpowiedzialna: Bartłomiej Raczyński

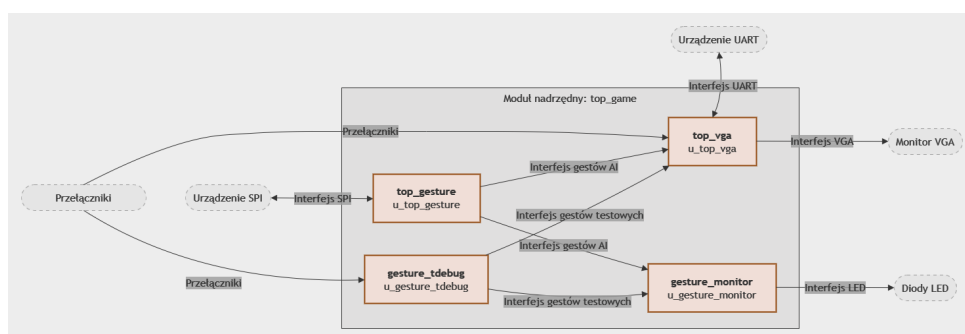
4.1.1 Schemat blokowy

⚠ Uwaga

Schemat blokowy **nie jest** schematem z Vivado! Nie zawiera sygnałów, tylko interfejsy (grupy sygnałów). Pokazuje moduły składowe oraz łączące je interfejsy.



Rysunek 1: Schemat blokowy całego projektu



Rysunek 2: Schemat blokowy modułu top

Zasady projektowania interfejsów:

- Interfejsy dwukierunkowe rozbijamy na 2 interfejsy jednokierunkowe.
- Nazwa interfejsu stanowi prefiks nazw sygnałów składowych.
- W interfejsach **nie** uwzględniamy sygnałów globalnych (clk, rst).

4.1.2 Porty

a) spi_in – spi rx, input

Nazwa portu	Opis
spi_in_miso	szeregowe wejście danych z czujnika gestów (Master In Slave Out)

b) spi_out – spi tx, output

Nazwa portu	Opis
spi_out_mosi	szeregowe wyjście danych do czujnika gestów (Master Out Slave In)
spi_out_sclk	sygnał zegara magistrali SPI
spi_out_cs_n	sygnał wyboru układu SPI (Chip Select), aktywny w stanie niskim

c) uart_in – uart rx, input

Nazwa portu	Opis
uart_in_rx	wejście odbiornika UART (odbieranie danych z drugiego układu FPGA)

d) uart_out – uart tx, output

Nazwa portu	Opis
uart_out_tx	wyjście nadajnika UART (wysyłanie danych do drugiego układu FPGA)

e) sw – switches, input

Nazwa portu	Opis
sw_master	wybór trybu działania układu (1 = Master, 0 = Slave)
sw_val[14:12]	przełączniki do debugowania gry

f) vga – vga_out, output

Nazwa portu	Opis
vga_vs	sygnał synchronizacji pionowej VGA
vga_hs	sygnał synchronizacji poziomej VGA
vga_r[3:0]	4-bitowy sygnał składowej koloru czerwonego
vga_g[3:0]	4-bitowy sygnał składowej koloru zielonego
vga_b[3:0]	4-bitowy sygnał składowej koloru niebieskiego

g) led – led_out, output

Nazwa portu	Opis
led_out[2:0]	wyjście na diody LED, sygnalizujące zdekodowany gest

4.1.3 Interfejsy

a) vif – vga_if (potok wideo pomiędzy modułami graficznymi)

Nazwa sygnału	Opis
vif_hcount[10:0]	horyzontalna pozycja piksela (licznik kolumn)
vif_vcount[10:0]	wertykalna pozycja piksela (licznik wierszy)
vif_hsync	wewnętrzny sygnał synchronizacji poziomej
vif_vsync	wewnętrzny sygnał synchronizacji pionowej
vif_hblnk	sygnał wygaszania poziomego (poza widocznym obszarem ekranu)
vif_vblnk	sygnał wygaszania pionowego (poza widocznym obszarem ekranu)
vif_rgb[11:0]	12-bitowa wartość koloru piksela (po 4 bity na kanały RGB)

b) gesture_out – gesture to vga/monitor (przesyłanie rozpoznanego gestu)

Nazwa sygnału	Opis
gesture_out	aktualnie wyznaczony gest (typ wyliczeniowy: NOTHING, KNOCK, SWIPE_RIGHT)

c) f2d – fsm to datapath (sterowanie przepływem gry Blackjack)

Nazwa sygnału	Opis
f2d_deal_enable	sygnał uruchamiający fazę początkowego rozdania kart
f2d_p0_turn	sygnał informujący o trwającej turze gracza 1
f2d_p1_turn	sygnał informujący o trwającej turze gracza 2
f2d_dealer_turn	sygnał informujący o trwającej turze krupiera

d) d2f – datapath to fsm (informacje zwrotne do maszyny stanów)

Nazwa sygnału	Opis
d2f_p0_bust	flaga informująca o przekroczeniu 21 punktów przez gracza 1
d2f_p1_bust	flaga informująca o przekroczeniu 21 punktów przez gracza 2
d2f_deal_done	flaga informująca o pomyślnym zakończeniu rozdania początkowego
d2f_dealer_done	flaga informująca o zakończeniu dobierania kart przez krupiera

e) d2c – datapath to card_drawing (żądanie losowania)

Nazwa sygnału	Opis
d2c_card_req	żądanie wygenerowania nowej (kolejnej) karty ze sprzętowego LFSR

f) c2d – card_drawing to datapath (wynik losowania z LFSR)

Nazwa sygnału	Opis
c2d_card_valid	potwierdzenie dostępności nowo wylosowanej karty
c2d_card_value[5:0]	wartość wylosowanej karty (0-51, mapowana wewnętrznie na kolor i figurę)

g) u2f – uart to fsm (komendy sieciowe przesyłane do FSM)

Nazwa sygnału	Opis
u2f_req_hit	żądanie dobrania karty (Hit) zgłoszone zdalnie przez moduł Slave
u2f_req_stand	żądanie zakończenia tury (Stand) zgłoszone zdalnie przez moduł Slave

h) u2d – uart to datapath (synchronizacja kart w trybie Slave)

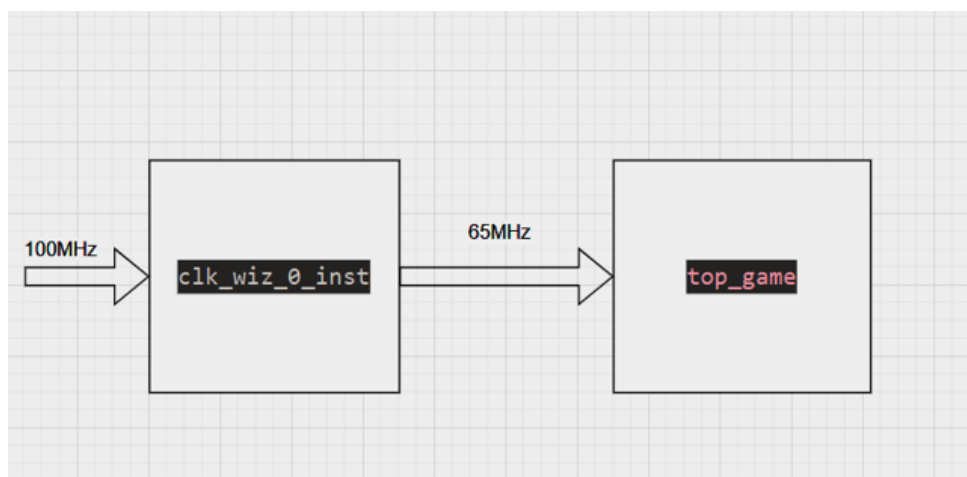
Nazwa sygnału	Opis
u2d_card_valid	flaga nadejścia nowej karty odebranej przez UART
u2d_card_val[5:0]	odebrana przez UART wartość przydzielonej karty
u2d_card_dst[1:0]	dwubitowy identyfikator odbiorcy karty (Gracz 1, Gracz 2, Krupier)

4.2 Rozprowadzenie sygnału zegara

Osoba odpowiedzialna: Bartłomiej Raczyński

W całym projekcie korzystamy z **jednego zegara** wygenerowanego za pomocą IP Core w Vivado:

65 MHz



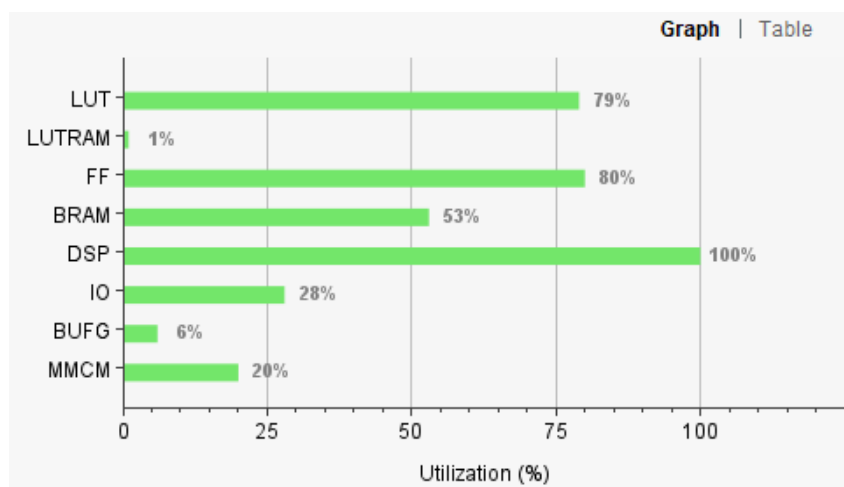
Rysunek 3: Rozprowadzenie sygnału zegara

5 Implementacja

5.1 Lista zignorowanych ostrzeżeń Vivado

Identyfikator	Liczba	Uzasadnienie
[Synth 8-7129]	99	Porty (sygnały konfiguracyjne z modułu AI myproject oraz nieużywane bity kolorów z vga_in) celowo pozostawione niepodłączone; syntezytor poprawnie je ignoruje.
[Synth 8-7080]	1	Ostrzeżenie wewnętrzne Vivado o braku wielowątkowej syntezy fragmentu kodu. Wpływa jedynie na czas kompilacji, nie na wygenerowany układ.
[Synth 8-6014]	1	Usunięty nieużywany element sekwencyjny (target_bit_reg). Poprawna optymalizacja niewykorzystanego kodu.
[Place 30-770]	1	Wysoka użycie rejestrów – upakowanie do 8 rejestrów na Slice. Placement poprawny; przy dodatnim WNS ostrzeżenie nie wpływa na stabilność.
[Power 33-332]	1	Długi stan aktywny globalnego resetu w analizatorze poboru mocy. Znaczenie wyłącznie w symulacjach energetycznych.
[Synth 8-3323]	1	Niedobór bloków DSP – Vivado zaimplementowało logikę z bramek LUT. Działanie poprawne.
[Synth 8-3936]	2	Nieużywane starsze bity rejestrów (title_y_diff_reg, title_x_diff_reg) zostały przycięte przez syntezytor, redukując użycie zasobów.
[Synth 8-7137]	3	Rejestr mem_reg posiada sygnały set i reset o tym samym priorytecie. Jednoczesne wystąpienie kontrolowane przez logikę sterującą; brak błędów w działaniu.
[Synth 8-4767]	1	Pamięć mem_reg zaimplementowana z użyciem rejestrów rozproszonych (Distributed RAM/LUT) zamiast BRAM – pożądane zachowanie dla małego bufora.

5.2 Wykorzystanie zasobów



Rysunek 4: Wykorzystanie zasobów FPGA

Resource	Utilization	Available	Utilization %
LUT	16518	20800	79.41
LUTRAM	19	9600	0.20
FF	33088	41600	79.54
BRAM	26.50	50	53.00
DSP	90	90	100.00
IO	30	106	28.30
BUFG	2	32	6.25
MMCM	1	5	20.00

Rysunek 5: Szczegółowe Wykorzystanie zasobów FPGA

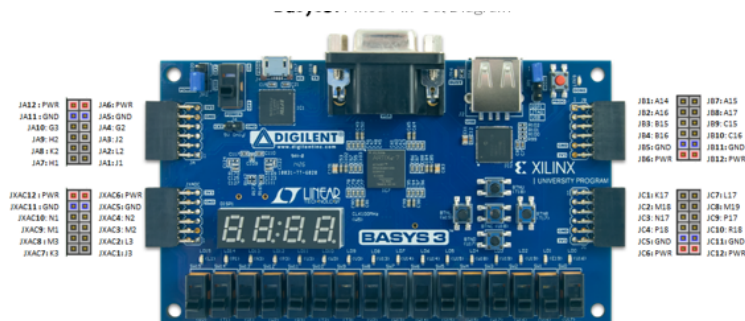
5.3 Marginesy czasowe

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 0,065 ns	Worst Hold Slack (WHS): 0,053 ns	Worst Pulse Width Slack (WPWS): 3,000 ns
Total Negative Slack (TNS): 0,000 ns	Total Hold Slack (THS): 0,000 ns	Total Pulse Width Negative Slack (TPWS): 0,000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 69615	Total Number of Endpoints: 69615	Total Number of Endpoints: 33221

All user specified timing constraints are met.

Rysunek 6: Marginesy czasowe setup i hold

6 Konfiguracja sprzętu



Rysunek 7: Schemat płytki Basys3 z pinoutem

6.1 Schemat połączenia płytek Basys3 w trybie multiplayer

Protokół komunikacji: UART

Płytką A	Kierunek	Płytką B
JXAC3 (TX)	→	JXAC4 (RX)
JXAC4 (RX)	←	JXAC3 (TX)

Sygnały TX i RX podpinamy **na przemian** (TX↔RX).

6.2 Podłączenie urządzeń peryferyjnych

Sygnal SPI	Pin Pmod	Opis
CS_n	JC1	Chip Select (aktywny niski)
SCLK	JC2	Zegar SPI
MOSI	JC3	Master Out Slave In
MISO	JC4	Master In Slave Out
GND	GND	Masa
VCC	PWR (3.3V)	Zasilanie

6.3 Konfiguracja przełączników

Przełącznik	Nazwa	Opis
SW15	SW_Master	Wybór roli płytki: góra = Master, dół = Slave. Jedna płytka zawsze musi być Masterem.
SW14	IMU_bypass	Góra = tryb ręczny (przełączniki zamiast czujnika IMU); Dół = odczyt z czujnika LSM6DSO.
SW13	KNOCK	(Aktywny gdy SW14=góra) Symulacja gestu podwójne puknięcie .
SW12	SWIPE	(Aktywny gdy SW14=góra) Symulacja gestu przesunięcie w prawo .

7 Film

Link do pobrania filmu:

<https://drive.google.com/drive/folders/1iLhEH2wCeFq1-hwKnAxRqGB5PSB9yz98?usp=sharing>

Film przedstawia działanie systemu: obsługę gestów, renderowanie VGA oraz tryb multiplayer.